

GTU ROBOTICS CLUB

How to perform Nvidia-deepstream feature in host system :

1. Go through docker deployment with TensorRT framework

2. In the host system:

Cmd : docker login nvcr.io

It will ask you Username , Password

3. To find the username and Password

https://docs.nvidia.com/metropolis/deepstream/dev-guide/text/DS_docker_containers.html

As you enter on the website go through this setup docker, setup nvidia container toolkit on host system

Prerequisites

1. Install `docker-ce` by following the [official instructions](#).

Once you have installed `docker-ce`, follow the [post-installation steps](#) to ensure that the docker can be run without `sudo`.

2. Install `nvidia-container-toolkit` by following the [install-guide](#).

3. Get an [NGC](#) account and API key:

- Go to NGC and search the **DeepStream** in the **Container** tab. This message is displayed: "Sign in to access the PULL feature of this repository".
- Enter your Email address and click **Next**, or click **Create an Account**.
- Choose your organization when prompted for **Organization/Team**.
- Click **Sign In**.

4. Log in to the NGC docker registry (`nvcr.io`) using the command `docker login nvcr.io` and enter the following credentials:

- ```
a. Username: "$oauthtoken"
b. Password: "YOUR_NGC_API_KEY"
```

where `YOUR_NGC_API_KEY` corresponds to the key you generated from step 3.

4. Go to the NGC Account

<https://catalog.ngc.nvidia.com/>

Make the account on right side corner

## Steps to get your API Key from NVIDIA NGC

### 1. Log in to NGC

Go to <https://ngc.nvidia.com> and sign in with your NVIDIA account.

### 2. Go to Setup → API Key

- On the top-right corner, click your profile icon.
- Select **Setup**.
- In the left menu, click **API Key**.

### 3. Generate API Key

- If you don't already have one, click **Generate API Key**.
- It will show you a long string (about 80–100 characters).
- Copy and save it somewhere safe (this is your “password” for Docker login).

The screenshot displays the NVIDIA NGC API Keys management interface. The left sidebar contains navigation links: Dashboard, Audit, Organization Profile, Teams, Usage, Activate Subscription, and Subscriptions. The main content area is titled 'Setup > API Keys' and 'API Keys'. A green button labeled 'Generate Personal Key' is in the top right. Below this is a table of 'Personal Keys' showing two active keys for the user 'Purushottam Jha'. The table columns are Key ID, Key Name, Expiration, Key Value, Status, and Actions. Below the table, a 'Display' dropdown is set to '25 results per page'. A section titled 'NGC API Keys' provides information about the two types of keys: Personal API Key and Legacy API Key.

| Key ID                               | Key Name        | Expiration | Key Value      | Status | Actions |
|--------------------------------------|-----------------|------------|----------------|--------|---------|
| faec91e1-cf75-4d12-91c9-5436c52aac3d | Purushottam Jha | 09/24/2026 | nvapi-*****Y9s | ACTIVE | ⋮       |
| 263ee34e-2b54-4d47-9dad-2d648f7e3ed0 | Purushottam Jha | 09/27/2026 | nvapi-*****BcU | ACTIVE | ⋮       |

**NGC API Keys**  
NVIDIA NGC API keys are required to authenticate to NGC services using NGC CLI, Docker CLI, or API communication. NVIDIA NGC supports two types of API keys.

**Personal API Key**  
Any user who is a member of an NGC org can generate Personal API Keys. These keys are tied to the user's lifecycle within the NGC org and can access up to the permissions and services assigned to the user. During the key generation steps, users can configure which NGC services are accessible by the API key and the time-to-live from one hour to 'never expires'.

**Legacy API Key**  
This is the original type of API key available in NGC since its inception. This type allows you to create only one 'API key' at a time. Generating a new key automatically revokes the previous one, as they cannot be rotated. The active key immediately becomes invalid when you create a new key.  
NVIDIA will continue to support this key type for services that have not transitioned to the next-generation API keys. However, we encourage customers to migrate to our next-generation API keys when possible.

**NGC CLI**

That page you've opened is the **NGC API Keys** page where your **Personal API Key** is shown.

- The **Key Value** (hidden with `nvapi-*****`) is the **long API key** you need.
- You can copy it by clicking on the little **copy icon** (next to the key value).

When you see `nvapi-*****Y9s`, that's just the **masked preview**, not the real key. Docker login will always fail with that.

Here's how you get the **full API key**:

## 🔑 Steps to copy the full NGC API Key

1. Go to your **NGC API Keys** page (the screenshot you sent).
2. On the right side of your key row, click the **three vertical dots (:)** under "Actions".
3. Select **Copy API Key** (or **Download Key** depending on UI).
  - This will copy the full `nvapi-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx` key to your clipboard.
4. Paste it into a text editor temporarily to confirm it's long (around 80+ characters).

### Sometimes you dont get copy API Key option so how to solve it ?

 **How to fix:**

1. Go to your **NGC account** → **Personal Keys page**.
2. Click on the three dots (⋮) under **Actions** → **Rotate Personal Key**.
3. This will generate a **new key** (the old one will be revoked).
4. When the new key is generated, **immediately copy it and store it** somewhere safe (like a password manager, or in `~/.ngc/config`).
5. Then use it in your Docker login:

```
purushottam@purushottam-IdeaPad-Gaming-3-15IHU6: ~
purushottam@purushottam-IdeaPad-Gaming-3-15IHU6:~$ docker login nvcr.io
Username: $oauthtoken
Password:
WARNING! Your password will be stored unencrypted in /home/purushottam/.docker/c
onfig.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credential-stores

Login Succeeded
purushottam@purushottam-IdeaPad-Gaming-3-15IHU6:~$
```

Now go with ahead setup everything :

```
bash

docker login nvcr.io
Username: $oauthtoken
Password: <paste your new key here>
```

Then you got this 👍

Your Login succeed finally:

## A. Configuration on Host system:

a.Ubuntu :22.04.1

b.Install DeepStream 7.0 Dependencies: Install the exact required versions

- **NVIDIA Driver:** 535.161.08
- **CUDA Toolkit:** 12.2
- **TensorRT:** 8.6.1.6
- **Cudnn:** 8.9.7.29-1+cuda12.2
- **Docker:** Which is already done in NGC Catalog Login Procedure

### Final Installation Steps(Only for tensorRT and Cudnn)

#### 1. Clean Up Previous Repository Setups

First, let's remove the configurations for the local repositories to avoid any conflicts.

Bash

```
sudo rm /etc/apt/sources.list.d/cuda-*.list
```

```
sudo rm /etc/apt/preferences.d/cuda-repository-pin-600
```

```
sudo apt-get clean
```

*(It is okay if these commands report that a file does not exist.)*

#### 2. Set Up the NVIDIA Network Repository

This step will configure your system to download packages directly from NVIDIA's official online repository.

Bash

```
wget https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64/cuda-keyring_1.1-1_all.deb
```

```
sudo dpkg -i cuda-keyring_1.1-1_all.deb
```

```
sudo apt-get update
```

#### 3. Install cuDNN

TensorRT depends on the cuDNN library, so we must install it first. Now that the network repository is set up, **apt** will be able to find it.

```
sudo apt-get install -y libcudnn8 libcudnn8-dev
```

## 4.install the TensorRT Packages

Execute the following command. It is an expanded version of the previous command that includes the specific header files that were causing the error.

Cmd :

```
sudo apt-get install -y --no-install-recommends \
 libnvinfer-headers-dev=8.6.1.6-1+cuda12.0 \
 libnvinfer-headers-plugin-dev=8.6.1.6-1+cuda12.0 \
 libnvinfer8=8.6.1.6-1+cuda12.0 \
 libnvinfer-dev=8.6.1.6-1+cuda12.0 \
 libnvinfer-plugin8=8.6.1.6-1+cuda12.0 \
 libnvinfer-plugin-dev=8.6.1.6-1+cuda12.0 \
 libnvparsers8=8.6.1.6-1+cuda12.0 \
 libnvparsers-dev=8.6.1.6-1+cuda12.0 \
 libvonnxparsers8=8.6.1.6-1+cuda12.0 \
 libvonnxparsers-dev=8.6.1.6-1+cuda12.0 \
 python3-libnvinfer=8.6.1.6-1+cuda12.0
```

Final check:

```
purushottam@purushottam-IdeaPad-Gaming-3-151H06: $ nvcc --version
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2023 NVIDIA Corporation
Built on Tue Jun 13 19:10:58 PDT 2023
Cuda compilation tools, release 12.2, V12.2.91
Build cuda_12.2.r12.2/compiler.32965470_0
purushottam@purushottam-IdeaPad-Gaming-3-151H06: $ nvidia-smi
Sat Oct 11 11:13:31 2025
+-----+
| NVIDIA-SMI 535.161.08 Driver Version: 535.161.08 CUDA Version: 12.2 |
+-----+-----+
GPU Name Persistence-M	Bus-Id Disp.A	Volatile Uncorr. ECC
Fan Temp Perf Pwr:Usage/Cap	Memory-Usage	GPU-Util Compute M.
	MIG M.	
+-----+-----+		
0 NVIDIA GeForce RTX 3050 ... Off	00000000:02:00:00 Off	
N/A 42C P0 N/A / 60W	7MiB / 4096MiB	0% Default
+-----+-----+		
+-----+		
Processes:		
GPU GI CI PID Type Process name GPU Memory		
ID ID	Usage	
+-----+-----+		
0 N/A N/A 1898 G /usr/lib/xorg/Xorg 4MiB		
+-----+
purushottam@purushottam-IdeaPad-Gaming-3-151H06: $ dpkg -l | grep tensorrt
ii nv-tensorrt-local-repo-ubuntu2204-8.6.1-cuda-12.0-1.0-1
purushottam@purushottam-IdeaPad-Gaming-3-151H06: $ dpkg -l | grep nvinfer
ii libnvinfer-dev 8.6.1.6-1+cuda12.0
ii libnvinfer-headers-dev 8.6.1.6-1+cuda12.0
ii libnvinfer-headers-plugin-dev 8.6.1.6-1+cuda12.0
ii libnvinfer-plugin-dev 8.6.1.6-1+cuda12.0
ii libnvinfer-plugin8 8.6.1.6-1+cuda12.0
ii libnvinfer-vc-plugin8 8.6.1.6-1+cuda12.0
ii libnvinfer8 8.6.1.6-1+cuda12.0
ii python3-libnvinfer 8.6.1.6-1+cuda12.0
purushottam@purushottam-IdeaPad-Gaming-3-151H06: $ docker --version
Docker version 27.5.1, build 27.5.1-0ubuntu3-22.04.2
purushottam@purushottam-IdeaPad-Gaming-3-151H06: $ docker info | grep -i runtime
ERROR: permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Get "http://%2Fvar%2Frun%2Fdocker.sock/v1.47/info": dial unix /var/run/docker.sock: connect: permission denied
errors pretty printing info
purushottam@purushottam-IdeaPad-Gaming-3-151H06: $ sudo docker info | grep -i runtime
[sudo] password for purushottam:
runtime: io.containerd.runc.v2 nvidia runc
purushottam@purushottam-IdeaPad-Gaming-3-151H06: $ dpkg -l | grep cudnn
ii libcudnn8 8.9.7.29-1+cuda12.2
ii libcudnn8-dev 8.9.7.29-1+cuda12.2
and64 nv-tensorrt-local repository configuration files
and64 TensorRT development libraries
and64 TensorRT development headers
and64 TensorRT plugin headers
and64 TensorRT plugin libraries
and64 TensorRT plugin libraries
and64 TensorRT vc-plugin library
and64 TensorRT runtime libraries
and64 Python 3 bindings for TensorRT standard runtime
and64 cudNN runtime libraries
and64 cudNN development libraries and headers
```

## c.Set the Nvidia Container

Find the best image ever for the system on the website  
[https://docs.nvidia.com/metropolis/deepstream/dev-guide/text/DS\\_docker\\_containers.html](https://docs.nvidia.com/metropolis/deepstream/dev-guide/text/DS_docker_containers.html)

For this system:

**sudo docker pull [nvcr.io/nvidia/deepstream:7.0-gc-triton-devel](https://docs.nvidia.com/metropolis/deepstream/dev-guide/text/DS_docker_containers.html)**

Enter the container with all access

Cmd:

```
sudo docker run -it --name deepstream_container \

--gpus all \

--device=/dev/bus/usb:/dev/bus/usb \

--device=/dev/video0:/dev/video0 \

--privileged \

-e DISPLAY=${DISPLAY} \

-v /tmp/.X11-unix:/tmp/.X11-unix \

nvcr.io/nvidia/deepstream:7.0-gc-triton-devel /bin/bash
```

```
purushottan@purushottan-IdeaPad-Gaming-3-151H06:~$ sudo docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
purushottan/ultralytics_d455 latest 19c4704b47fd 8 days ago 62.2GB
ultralytics_d455 latest 19c4704b47fd 8 days ago 62.2GB
ultralytics_backup latest 83c8bcb1c271 2 weeks ago 56.1GB
nvcr.io/nvidia/deepstream 7.0-gc-triton-devel 7afec3e548f3 17 months ago 20.9GB
purushottan@purushottan-IdeaPad-Gaming-3-151H06:~$ sudo docker ps -a
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
2dd929a0c713 ultralytics_d455 "/bin/bash" 8 days ago Exited (0) 8 days ago ultralytics_latest_container
0694fdbd00f0 ultralytics_backup "/bin/bash" 2 weeks ago Exited (0) 8 days ago ultralytics_container
purushottan@purushottan-IdeaPad-Gaming-3-151H06:~$ sudo docker run -it --name deepstream_container \
--gpus all \
--device=/dev/bus/usb:/dev/bus/usb \
--device=/dev/video0:/dev/video0 \
--privileged \
-e DISPLAY=${DISPLAY} \
-v /tmp/.X11-unix:/tmp/.X11-unix \
nvcr.io/nvidia/deepstream:7.0-gc-triton-devel /bin/bash

=====
DeepStreamSDK 7.0.0
=====

*** LICENSE AGREEMENT ***
By using this software you agree to fully comply with the terms and conditions
of the License Agreement. The License Agreement is located at
/opt/nvidia/deepstream/deepstream/LicenseAgreement.pdf. If you do not agree
to the terms and conditions of the License Agreement do not use the software.

=====
== Triton Inference Server ==
=====

NVIDIA Release 23.10 (build 72127154)
Triton Server Version 2.39.0

Copyright (c) 2018-2023, NVIDIA CORPORATION & AFFILIATES. All rights reserved.
Various files include modifications (c) NVIDIA CORPORATION & AFFILIATES. All rights reserved.

This container image and its contents are governed by the NVIDIA Deep Learning Container License.
By pulling and using the container, you accept the terms and conditions of this license:
https://developer.nvidia.com/ngc/nvidia-deep-learning-container-license

root@7d5a482009b9:/opt/nvidia/deepstream/deepstream-7.0# nvidia-smi
```

Check Everything Step by Step

```

root@7d5a482009b9:/opt/nvidia/deepstream/deepstream-7.0# ls /dev/video*
/dev/video0
root@7d5a482009b9:/opt/nvidia/deepstream/deepstream-7.0# lsusb
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 001 Device 003: ID 048d:c966 Integrated Technology Express, Inc. ITE Device(8176)
Bus 001 Device 002: ID 5986:212b Acer, Inc Integrated Camera
Bus 001 Device 004: ID 8087:0026 Intel Corp. AX201 Bluetooth
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
root@7d5a482009b9:/opt/nvidia/deepstream/deepstream-7.0# nvidia-smi
Sat Oct 11 09:54:35 2025
+-----+
| NVIDIA-SMI 535.161.08 Driver Version: 535.161.08 CUDA Version: 12.2 |
+-----+
| GPU Name Persistence-M | Bus-Id Disp.A | Volatile Uncorr. ECC |
| Fan Temp Perf Pwr:Usage/Cap | Memory-Usage | GPU-Util Compute M. |
|=====+=====+
| 0 NVIDIA GeForce RTX 3050 Off | 00000000:02:00:0 Off | 0% N/A |
| N/A 42C P3 N/A / 30W | 7MiB / 4096MiB | 0% Default |
|=====+=====+
+-----+

Processes:
+-----+
| GPU GI CI PID Type Process name GPU Memory |
| ID ID Usage |
+-----+
root@7d5a482009b9:/opt/nvidia/deepstream/deepstream-7.0# nvcc --version
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2023 NVIDIA Corporation
Built on Tue_Aug_15_22:02:13_PDT_2023
Cuda compilation tools, release 12.2, V12.2.140
Build cuda_12.2.r12.2/compiler.33191640_0

```

```

root@7d5a482009b9:/opt/nvidia/deepstream/deepstream-7.0# deepstream-app --version-all
Glib (gthread-posix.c): Unexpected error from C library during 'pthread_setspecific': Invalid argument. Aborting.
(gst-plugin-scanner:869): GStreamer-WARNING **: 09:55:01.724: Failed to load plugin '/usr/lib/x86_64-linux-gnu/gstreamer-1.0/deepstream/libnvds_gst_udp.so': librivernax.so.1: cannot open shared object file: No such file or directory
(gst-plugin-scanner:869): GStreamer-WARNING **: 09:55:01.968: Failed to load plugin '/usr/lib/x86_64-linux-gnu/gstreamer-1.0/libgstmpeg2enc.so': libmpeg2encpp-2.1.so.0: cannot open shared object file: No such file or directory
(gst-plugin-scanner:869): GStreamer-WARNING **: 09:55:02.079: Failed to load plugin '/usr/lib/x86_64-linux-gnu/gstreamer-1.0/libgstchromaprint.so': libavcodec.so.58: cannot open shared object file: No such file or directory
libva info: VA-API version 1.14.0
libva info: Trying to open /usr/lib/x86_64-linux-gnu/dri/LIBDRM_drv_video.so
libva info: va_openDriver() returns -1
libva info: Trying to open /usr/lib/x86_64-linux-gnu/dri/LIBGLX_drv_video.so
libva info: va_openDriver() returns -1
(gst-plugin-scanner:869): GStreamer-WARNING **: 09:55:02.256: Failed to load plugin '/usr/lib/x86_64-linux-gnu/gstreamer-1.0/libgstopenmpt.so': libmpg123.so.0: cannot open shared object file: No such file or directory
(gst-plugin-scanner:869): GStreamer-WARNING **: 09:55:02.295: Failed to load plugin '/usr/lib/x86_64-linux-gnu/gstreamer-1.0/libgstmpeg2dec.so': libmpeg2.so.0: cannot open shared object file: No such file or directory
(gst-plugin-scanner:869): GStreamer-WARNING **: 09:55:02.298: Failed to load plugin '/usr/lib/x86_64-linux-gnu/gstreamer-1.0/libgstmpg123.so': libmpg123.so.0: cannot open shared object file: No such file or directory
deepstream-app version 7.0.0
DeepStreamDK 7.0.0
CUDA Driver Version: 12.2
CUDA Runtime Version: 12.2
TensorRT Version: 8.6
cuDNN Version: 8.9
libNVWarp308 Version: 2.0.1d3
root@7d5a482009b9:/opt/nvidia/deepstream/deepstream-7.0#

```

Now the time to thrill

# 1.Video Check

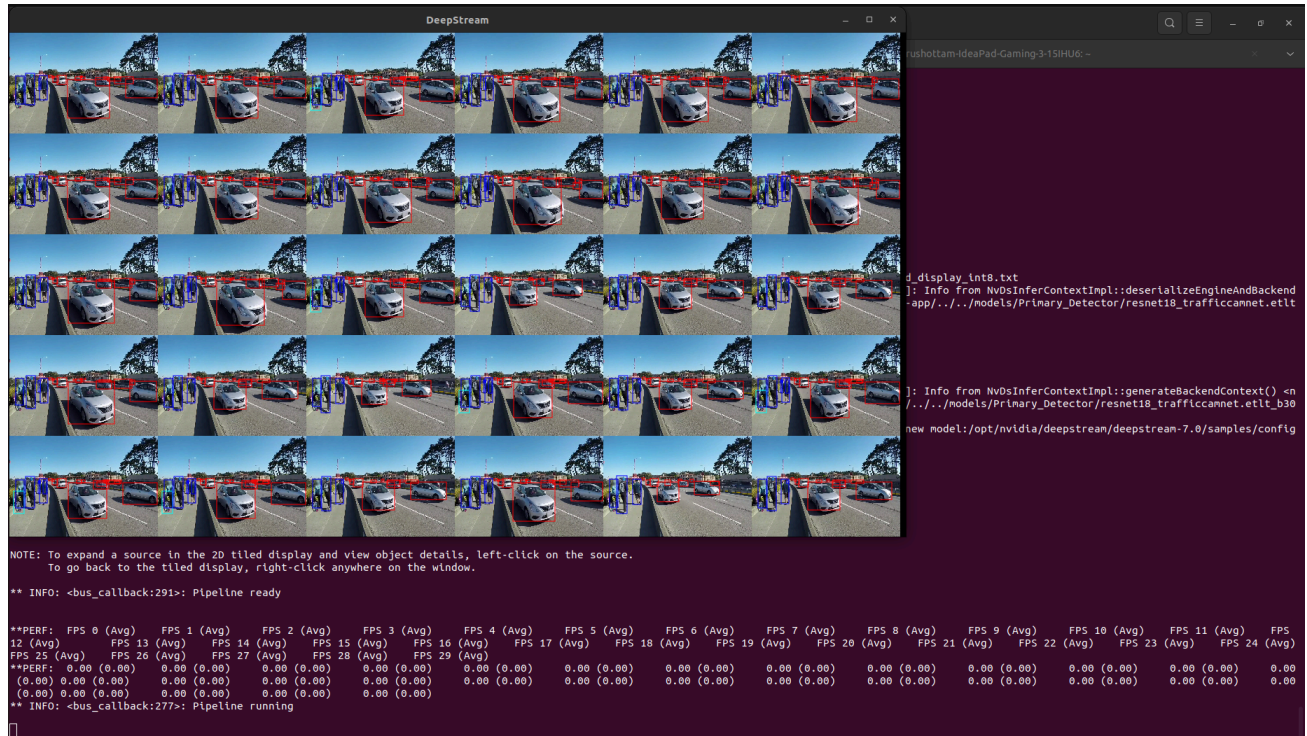
Test the sample video on the deepstream container

Cmd: `/opt/nvidia/deepstream/deepstream/user_additional_install.sh`

Cmd: `cd /opt/nvidia/deepstream/deepstream-7.0/samples/configs/deepstream-app`

Cmd: `deepstream-app -c source30_1080p_dec_infer-resnet_tiled_display_int8.txt`





How to edit file from 30 to 4 video stream

Cmd : `cp source30_1080p_dec_infer-resnet_tiled_display_int8.txt source4_1080p_dec_infer-resnet_int8.txt`

Cmd: : `nano source30_1080p_dec_infer-resnet_int8.txt`

Open the file

## How to make it 4 streams

You need to:

### 1. Reduce `[sourceX]` blocks

- Keep only `[source0]` and `[source1]` (or even just `[source0]` with `num-sources=4`).

### 2. Set `num-sources` to 4

`[source0]`

`enable=1`

`type=3`

`uri=file:///.../streams/sample_1080p_h264.mp4`

```
num-sources=4
```

### 3. Update `[streammux]` batch-size

```
[streammux]
```

```
batch-size=4
```

### 4. Update `[tilted-display]` rows/columns

- For 4 streams → 2x2 tile:

```
[tilted-display]
```

```
rows=2
```

```
columns=2
```

---

## Example of a minimal 4-stream config

```
[source0]
enable=1
type=3
uri=file:///../../streams/sample_1080p_h264.mp4
num-sources=4
gpu-id=0
cudadec-memtype=0

[streammux]
gpu-id=0
batch-size=4
width=1920
height=1080
enable-padding=0

[tilted-display]
enable=1
```

```
rows=2
columns=2
width=1280
height=720
gpu-id=0

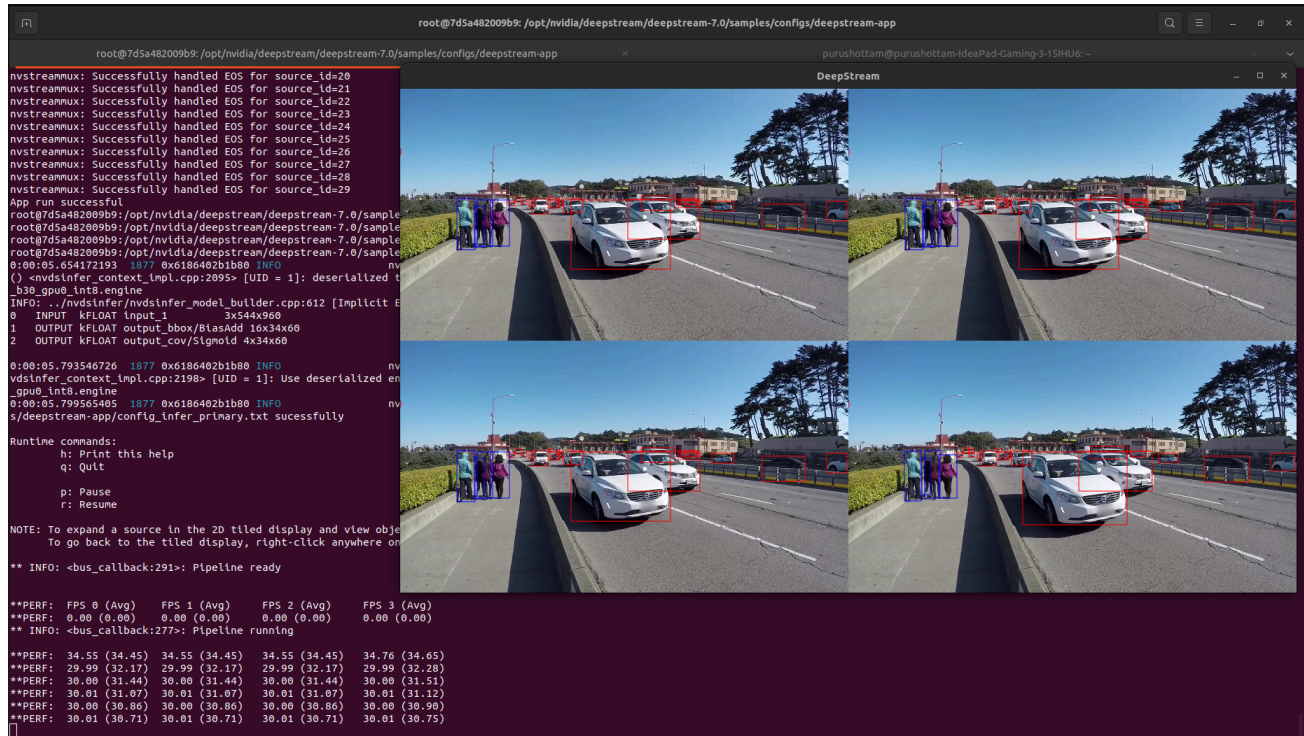
[primary-gie]
enable=1
gpu-id=0
model-engine-file=../../models/Primary_Detector/resnet18_trafficcarnet.etlt_b3
0_gpu0_int8.engine
batch-size=4
nvbuf-memory-type=0
```

This config will **run 4 videos instead of 30**, batch them together, and display in a 2x2 tile.

Run the file :

**Cmd: deepstream-app -c source4\_1080p\_dec\_infer-resnet\_int8.txt**

Output:



Source 30 means 30 videos display :

## 2.Start the camera in container

Test camera access inside container

a.Once inside container:

**ls /dev/video\***

- Should show **/dev/video0**
- Optionally, test with GStreamer:

**gst-launch-1.0 v4l2src device=/dev/video0 ! videoconvert ! autovideosink**

- You should see your webcam feed.

b.Check your camera configuration inside container

**Cmd: v4l2-ctl --list-formats-ext**

If not download v4l2,download it

**Cmd:apt update && apt install v4l-utils -y**

c.Make the file

**Cmd: nano source\_camera.txt**

d.edit the file according to your laptop webcam

```
[application]
enable-perf-measurement=1
perf-measurement-interval-sec=5

[streammux]
gpu-id=0
batch-size=1
width=1280
height=720
live-source=1
batched-push-timeout=40000

[source0]
enable=1
type=1 # V4L2 camera
uri=/dev/video0 # webcam device
num-sources=1
gpu-id=0
cudadec-memtype=0
camera-width=640
camera-height=480
camera-fps-n=30
camera-fps-d=1
gst-caps="image/jpeg, width=640, height=480, framerate=30/1"

[primary-gie]
enable=1
gpu-id=0
model-engine-file=../../models/Primary_Detector/resnet18_traf
ficcamnet.etlt_b30_gpu0_int8.engine
batch-size=1
nvbuf-memory-type=0
config-file=config_infer_primary.txt

[osd]
enable=1
```

```

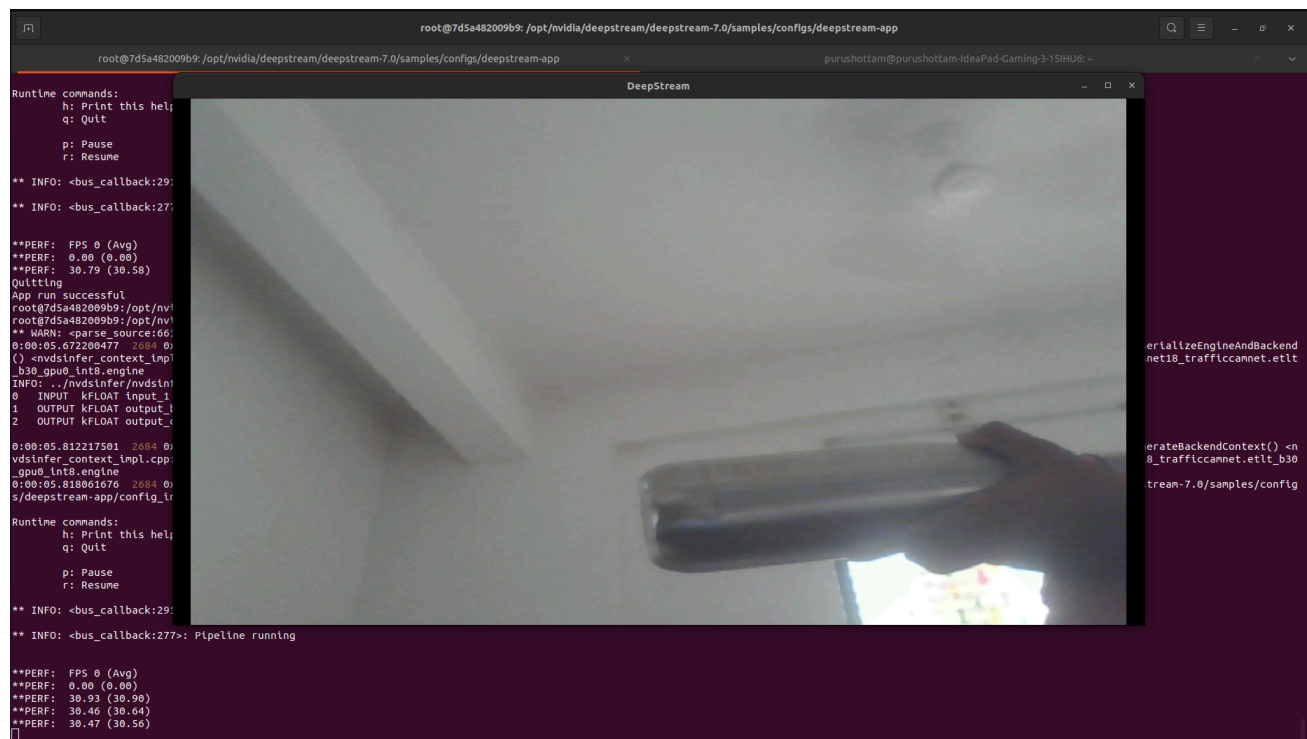
gpu-id=0
border-width=1
text-size=15
text-color=1;1;1;1;
text-bg-color=0.3;0.3;0.3;1
nvbuf-memory-type=0

[sink0]
enable=1
type=2 # EglSink / nv3dsink
sync=1
gpu-id=0
nvbuf-memory-type=0

```

Run the file :

Cmd:**deepstream-app -c source\_multi\_camera.txt**



### 3. Multiple camera feed 👍

Check : `ls /dev/video*`

Output: `/dev/video0 /dev/video1`

Inside the container

Cmd: `nano source_multi_camera.txt`

Code:

```
[application]
enable-perf-measurement=1
perf-measurement-interval-sec=5

[source0]
enable=1
type=1
uri=/dev/video0
gpu-id=0
cudadec-memtype=0
camera-width=640
camera-height=480
camera-fps-n=30
camera-fps-d=1
gst-caps="image/jpeg, width=640, height=480, framerate=30/1"

[source1]
enable=1
type=1
uri=/dev/video1
gpu-id=0
cudadec-memtype=0
camera-width=640
camera-height=480
camera-fps-n=30
camera-fps-d=1
gst-caps="image/jpeg, width=640, height=480, framerate=30/1"

[streammux]
gpu-id=0
live-source=1
batch-size=2 # number of cameras
width=1280
height=720
batched-push-timeout=40000
nvbuf-memory-type=0

[primary-gie]
```



```
enable=1
gpu-id=0
model-engine-file=../../models/Primary_Detector/resnet18_trafficcarnet.etlt_b30_gpu0_i
nt8.engine
batch-size=1
nvbuf-memory-type=0
config-file=config_infer_primary.txt

[osd]
enable=1
gpu-id=0
border-width=1
text-size=15
text-color=1;1;1;1;
text-bg-color=0.3;0.3;0.3;1
nvbuf-memory-type=0

[sink0]
enable=1
type=2 # EglSink / nv3dsink
sync=1
gpu-id=0
nvbuf-memory-type=0
```

Run the file:

**Cmd:deepstream-app -c source\_multi\_camera.txt**

One your laptop webcam open and second physical camera like intel d455

**Output** 👍

You saw two streams from two cameras.

## B. Now ready for Yolov8 + TensorRT deployment in Deepstream

### Step 1: Ensure the Custom Parser Library is Compiled

This is a one-time step. You must have the custom library file **libnvdsinfer\_custom\_impl\_Yolo.so**. If you have not compiled this yet, you must do it now inside the container.

```
(If you haven't already) Clone the required repository
git clone https://github.com/marcoslucianops/DeepStream-Yolo.git
cd DeepStream-Yolo

Set the correct CUDA version for DeepStream 7.0
export CUDA_VER=12.2

Compile the library
make -C nvdsinfer_custom_impl_Yolo clean && make -C nvdsinfer_custom_impl_Yolo
```

(Your current path:

```
cd /opt/nvidia/deepstream/deepstream-7.0/samples/configs/deepstream-app/Deepstream-Yolo)
```

1. Download and install all necessary library

(opencv, ultralytics, pyrealsense etc)

2. After download library run the cmd

**Cmd: yolo export model=yolov8n.pt format=onnx**

Now you have [yolov8n.pt](#) as well as yolov8n.onnx

3. Convert to .engine file

**Cmd: trtexec --onnx=yolov8n.onnx --saveEngine=yolov8n\_fp32.engine**

4. check through ls

Now you have [yolov8n.pt](#) as well as yolov8n.onnx as well as yolov8n\_fp32.engine

5. Now time to [best.pt](#)

Set the host to into the container

**Cmd:** `sudo docker cp /home/purushottam/Downloads/best.pt 7d5a482809b9:/opt/nvidia/deepstream/deepstream-7.0/samples/configs/deepstream-app/Deepstream-Yolo`

Convert to best.onnx

**Cmd: yolo export model=best.pt format=onnx**

Convert to best.engine

Cmd: `trtexec --onnx=best.onnx --saveEngine=best.engine`

6. Check ls

Everything is done now.

Final Output through After setup

```
root@7d5a482009b9: /opt/nvidia/deepstream/deepstream-7.0/samples/configs/deepstream-app/DeepStream-Yolo# ls
LICENSE.md config_infer_primary_dflne.txt config_infer_primary_yoloV2.txt config_infer_primary_yolonas_custom.txt labels.txt
README.md config_infer_primary_goldyolo.txt config_infer_primary_yoloV5.txt config_infer_primary_yolor.txt nvdsinfer_custom_impl_Yolo
best.engine config_infer_primary_ppyoloe.txt config_infer_primary_yoloV5u.txt config_infer_primary_yolov8.txt source_realsense.txt
best.onnx config_infer_primary_ppyoloe_plus.txt config_infer_primary_yoloV6.txt config_infer_primary_yolox.txt utils
best.pt config_infer_primary_rtdetr.txt config_infer_primary_yoloV7.txt config_infer_primary_yolox_legacy.txt yolov8n.onnx
config_infer_primary.txt config_infer_primary_rtmddet.txt config_infer_primary_yoloV8.txt config_infer_primary_yolox_legacy.txt yolov8n.pt
config_infer_primary_codetr.txt config_infer_primary_yolov10.txt config_infer_primary_yoloV9.txt deepstream_app_config.txt yolov8n_fp32.engine
config_infer_primary_danoyolo.txt config_infer_primary_yolov10.txt config_infer_primary_yolonas.txt deepstream_yolov8_realsense.txt docs
root@7d5a482009b9: /opt/nvidia/deepstream/deepstream-7.0/samples/configs/deepstream-app/DeepStream-Yolo#
```

You can check all files are ready

Step 2:Edit All The files

1.Cmd:nano deepstream\_yolov8\_realsense.txt

```
[application]
enable-perf-measurement=1
perf-measurement-interval-sec=5

[tiled-display]
enable=1
rows=1
columns=2
width=1280
height=720
gpu-id=0

[source0]
enable=1
type=1
camera-width=640
camera-height=480
camera-fps-n=30
camera-fps-d=1 # important!
camera-v4l2-dev-node=0

[source1]
enable=1
type=1
```

```
camera-width=640
camera-height=480
camera-fps-n=30
camera-fps-d=1 # important!
camera-v4l2-dev-node=4

[streammux]
gpu-id=0
batch-size=1
width=640
height=480
batched-push-timeout=40000

[primary-gie]
enable=1
gpu-id=0
config-file=config_infer_primary_yolov8.txt

[sink0]
enable=1
type=2
sync=0
gpu-id=0
```

## 2.Cmd:nano config\_infer\_primary\_yolov8.txt

```
[property]
gpu-id=0
net-scale-factor=0.0039215697906911373
onnx-file=yolov8n.onnx
model-engine-file=yolov8n_fp32.engine
network-mode=0
num-detected-classes=80 # Your number of classes
gie-unique-id=1
process-mode=1

--- THIS IS THE CRITICAL FIX ---
```

```
Point to the custom parser library you just compiled.
IMPORTANT: Adjust this path to be correct from where you run the app.
custom-lib-path=nvdsinfer_custom_impl_Yolo/libnvdsinfer_custom_impl_Yolo.so
Tell nvinfer which function to use for parsing YOLOv8 output.
parse-bbox-func-name=NvDsInferParseYolo

[class-attrs-all]
pre-cluster-threshold=0.25
Use Non-Maximum Suppression for clustering
cluster-mode=2
nms-iou-threshold=0.45
```

### 3.Cmd:nano source\_realsense.txt

```
source_realsense.txt

[application]
enable-perf-measurement=1
perf-measurement-interval-sec=5

[tiled-display]
enable=1
rows=1
columns=2
width=1280
height=720
gpu-id=0

[source0]
enable=1
type=1
camera-width=640
camera-height=480
camera-fps-n=30
camera-fps-d=1 # important!
camera-v4l2-dev-node=0

[source1]
enable=1
type=1
camera-width=640
camera-height=480
camera-fps-n=30
```

```

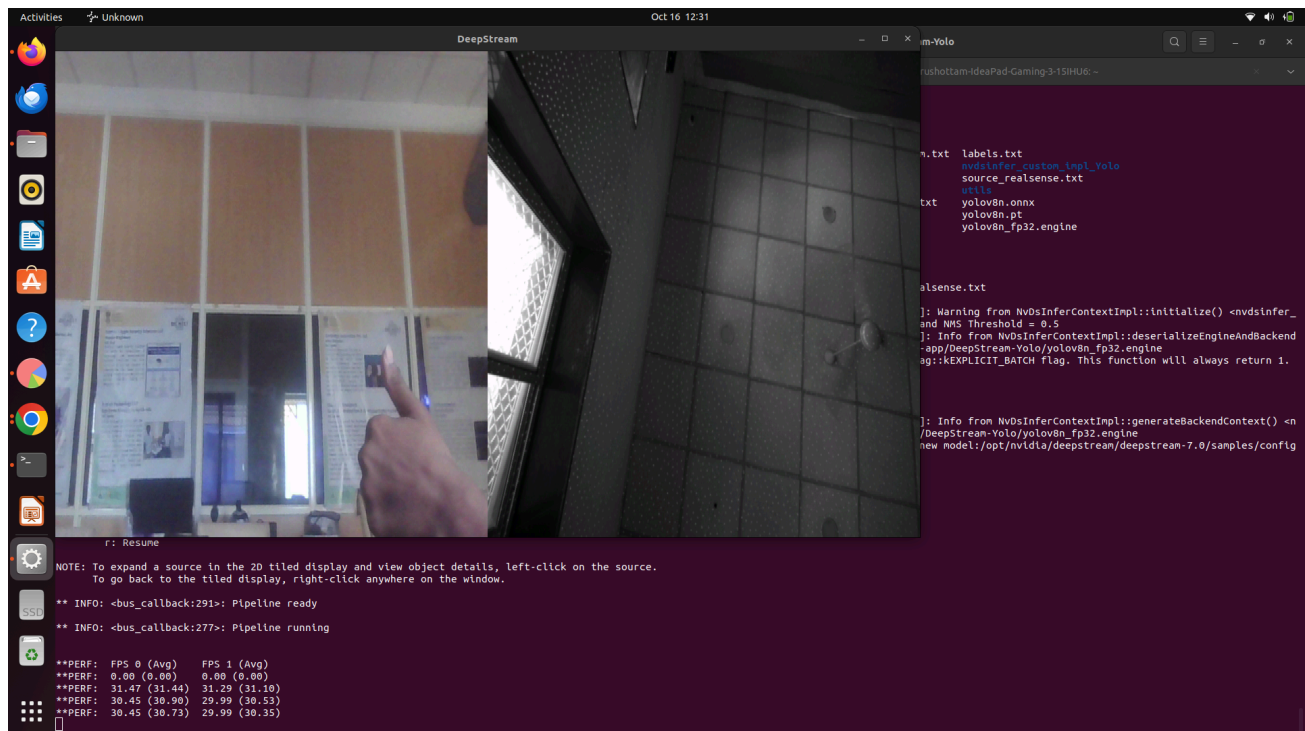
camera-fps-d=1 # important!
camera-v4l2-dev-node=4

[streammux]
gpu-id=0
live-source=1
batch-size=1
width=640
height=480
batched-push-timeout=40000

```

Final Output

Cmd: `deepstream-app -c deepstream_yolov8_realsense.txt`



See the magic

Now you check There are total two stream where first is Laptop Webcam And Camera is Intel D455 Realsense.

(Still the path

`cd /opt/nvidia/deepstream/deepstream-7.0/samples/configs/deepstream-app/Deepstream-Yolo)`

Step 3.Laptop webcam + IR cam + RGB Cam

**Cmd: v4l2-ctl --list-devices**

```
root@7d5a482089b9:/opt/nvidia/deepstream/deepstream-7.0/samples/configs/deepstream-app/deepStream-YoloW# v4l2-ctl --list-devices
Intel(R) RealSense(TM) Depth Ca (usb-0000:00:14.0-1):
/dev/video2
/dev/video3
/dev/video4
/dev/video5
/dev/video6
/dev/video7
/dev/media1
/dev/media2

Integrated Camera: Integrated C (usb-0000:00:14.0-6):
/dev/video8
/dev/video1
/dev/media8

root@7d5a482089b9:/opt/nvidia/deepstream/deepstream-7.0/samples/configs/deepstream-app/deepStream-YoloW#
```

Node 4 and 6 working

Means node 4 for IR and 6 for RGB(INTEL D455 REALSENSE)and 0 for laptop camera

Edit the source for both

**Cmd: nano Source\_realsense.txt**

**Cmd: nano deepstream\_yolov8\_realsense.txt**

```
[source0]
enable=1
type=1
camera-width=640
camera-height=480
camera-fps-n=30
camera-fps-d=1 # important!
camera-v4l2-dev-node=0

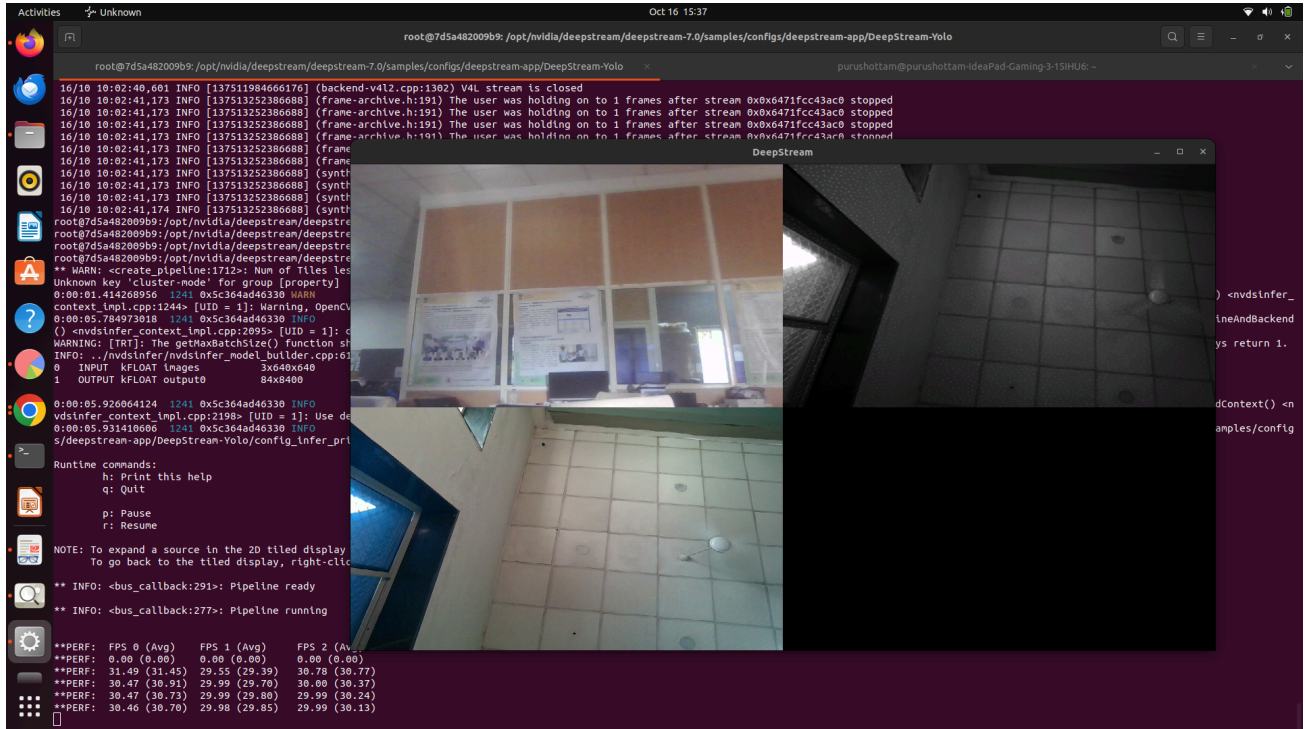
[source1]
enable=1
type=1
camera-width=640
camera-height=480
camera-fps-n=30
camera-fps-d=1 # important!
camera-v4l2-dev-node=4

[source2]
enable=1
type=1
camera-width=640
camera-height=480
camera-fps-n=30
camera-fps-d=1 # important!
camera-v4l2-dev-node=6
```

Save and exit

Now time to run the file

```
Cmd: deepstream-app -c deepstream_yolov8_realsense.txt
```



**✗ Depth stream** is black because DeepStream (and V4L2) cannot decode Intel RealSense depth formats like **Z16** or **Y12I**.